

1. Introdução ao Projeto

Neste tutorial você vai criar um **Sistema de Controle de Ponto e Horas Extras** usando **Java 17+** e **Spring Boot 3**. O sistema permitirá:

- Cadastrar funcionários com cargo e carga horária
- Importar registros de ponto via arquivo CSV
- Calcular automaticamente horas trabalhadas e horas extras
- Consultar relatórios via API REST ou navegando o sistema

Sistema de Controle de Horas com Java + Spring Boot

Linguagem	Java 17 ou superior
Framework	Spring Boot 3.x
Banco de dados	H2 (em memória) — sem instalação
Build tool	Maven (gerenciador de dependências)
IDE	IntelliJ IDEA Community (grátis) ou VS Code

■ Se nunca programou em Java, não se preocupe. Cada linha de código será explicada passo a passo neste tutorial.

✓ Estrutura de projeto Spring Boot

2. Instalando as Ferramentas

✓ Leitura de arquivos CSV

2.1 — Java (Cálculo de horas trabalhadas e extras)

- Acesse: <https://adoptium.net> e baixe o **JDK 17 LTS**
- Execute o instalador e clique em Next até finalizar
- Abra o Prompt de Comando (CMD) e verifique:

```
java -version
# Deve aparecer: openjdk version "17.x.x"
```

2.2 — IDE: IntelliJ IDEA

- Acesse: <https://www.jetbrains.com/idea/download>
- Baixe a versão **Community Edition** (gratuita)
- Instale normalmente seguindo o assistente

2.3 — Maven (opcional)

O IntelliJ já inclui Maven embutido. Mas se quiser usar pelo terminal:

- Acesse: <https://maven.apache.org/download.cgi>

- Baixe o zip, extraia em C:\maven e adicione ao PATH do sistema

```
mvn -version
# Deve aparecer: Apache Maven 3.x.x
```

■ Ferramentas prontas? Vamos criar o projeto!

3. Criando o Projeto Spring Boot

O **Spring Initializr** é um site que gera a estrutura inicial do projeto automaticamente.

Passo a passo:

- Acesse <https://start.spring.io> no seu navegador
- Preencha os campos conforme abaixo:

Project	Maven Project
Language	Java
Spring Boot	3.2.x (a versão mais recente estável)
Group	com.empresa
Artifact	controle-horas
Name	controle-horas
Packaging	Jar
Java version	17

Dependências — clique em **ADD DEPENDENCIES** e adicione:

- **Spring Web** — para criar a API REST
- **Spring Data JPA** — para salvar dados no banco
- **H2 Database** — banco de dados em memória
- **Lombok** — reduz código repetitivo
- **OpenCSV** — para ler arquivos CSV
- Clique em **GENERATE** — um arquivo .zip será baixado
- Extraia o zip em uma pasta, ex: C:\projetos\controle-horas
- Abra o IntelliJ IDEA > **Open** > selecione a pasta extraída
- Aguarde o IntelliJ baixar as dependências (barra de progresso na parte inferior)

■ A primeira vez pode demorar alguns minutos pois o Maven baixa as bibliotecas da internet.

4. Estrutura de Pastas do Projeto

Após abrir no IntelliJ, a estrutura de pastas ficará assim:

```
controle-horas/
├── src/
│   ├── main/
│   │   ├── java/com/empresa/controlehoras/
│   │   │   ├── model/           ← Entidades (tabelas do banco)
│   │   │   ├── repository/      ← Acesso ao banco de dados
│   │   │   ├── service/         ← Regras de negócio
│   │   │   ├── controller/      ← Endpoints da API
│   │   │   ├── dto/             ← Objetos de transferência
│   │   │   ├── ControleHorasApplication.java
│   │   └── resources/
│   │       ├── application.properties
│   │       └── data.sql          ← Dados iniciais (opcional)
│   └── pom.xml                  ← Dependências do projeto
```

O que é cada pasta?

model/	Define como os dados são estruturados (ex: Funcionario tem nome, cargo, horas)
repository/	Código que conversa com o banco de dados (buscar, salvar, deletar)
service/	Onde ficam as regras do negócio: calcular horas extras, validações, etc.
controller/	Define as URLs da API (ex: /api/funcionarios, /api/relatorio)
dto/	Objetos simples para enviar/receber dados pela API sem expor o banco

■ Crie as pastas no IntelliJ clicando com o botão direito em `java/com/empresa/controlehoras` > New > Package

5. Configurando o pom.xml

O **pom.xml** é o arquivo de configuração do Maven. Ele lista todas as bibliotecas que o projeto usa. Substitua o conteúdo pelo código abaixo:

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0">
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <groupId>org.springframework.boot</groupId>
```

```
<artifactId>spring-boot-starter-parent</artifactId>
<version>3.2.0</version>
</parent>
<groupId>com.empresa</groupId>
<artifactId>controle-horas</artifactId>
<version>1.0.0</version>
<properties>
  <java.version>17</java.version>
</properties>
<dependencies>
  <!-- Spring Web: cria a API REST -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
  <!-- Spring Data JPA: banco de dados -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
  </dependency>
  <!-- H2: banco em memória (sem instalar) -->
  <dependency>
    <groupId>com.h2database</groupId>
    <artifactId>h2</artifactId>
    <scope>runtime</scope>
  </dependency>
  <!-- Lombok: remove código repetitivo -->
  <dependency>
    <groupId>org.projectlombok</groupId>
    <artifactId>lombok</artifactId>
    <optional>true</optional>
  </dependency>
  <!-- OpenCSV: leitura de CSV -->
  <dependency>
    <groupId>com.opencsv</groupId>
    <artifactId>opencsv</artifactId>
    <version>5.8</version>
  </dependency>
</dependencies>
</project>
```

6. Configuração do Banco de Dados

Abra o arquivo `src/main/resources/application.properties` e cole:

```
# Nome do banco de dados H2
spring.datasource.url=jdbc:h2:mem:controlehoras
spring.datasource.driver-class-name=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=
```

```
# Criar tabelas automaticamente
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true

# Console web do H2 (para ver dados no navegador)
spring.h2.console.enabled=true
spring.h2.console.path=/h2-console

# Porta onde o servidor vai rodar
server.port=8080
```

■ Com o H2 habilitado, você pode ver os dados em tempo real acessando <http://localhost:8080/h2-console> no navegador enquanto o sistema roda.

7. Criando os Models (Entidades)

Os **Models** representam as tabelas do banco de dados. Cada campo da classe vira uma coluna na tabela.

7.1 — Cargo.java

Representa os cargos da empresa com a carga horária diária esperada. Crie em **model/Cargo.java**:

```
package com.empresa.controlehoras.model;

import jakarta.persistence.*;
import lombok.Data;

// @Entity = esta classe é uma tabela no banco
// @Data = Lombok gera getters, setters, toString automaticamente
@Entity
@Data
public class Cargo {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id; // chave primária auto-incrementada

    @Column(nullable = false, unique = true)
    private String nome; // ex: "Desenvolvedor", "Gerente"

    // Horas que devem ser trabalhadas por dia
    private double horasDiarias; // ex: 8.0
}
```

7.2 — Funcionario.java

Representa cada funcionário. Crie em **model/Funcionario.java**:

```
package com.empresa.controlehoras.model;

import jakarta.persistence.*;
import lombok.Data;

@Entity
@Data
public class Funcionario {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String nome;

    private String matricula; // ex: "EMP001"

    // Relacionamento: cada funcionário tem um cargo
    // @ManyToOne = muitos funcionários podem ter o mesmo cargo
    @ManyToOne
    @JoinColumn(name = "cargo_id")
    private Cargo cargo;
}
```

7.3 — RegistroPonto.java

Representa cada batida de ponto. Crie em **model/RegistroPonto.java**:

```
package com.empresa.controlehoras.model;

import jakarta.persistence.*;
import lombok.Data;
import java.time.LocalDate;
import java.time.LocalTime;

@Entity
@Data
public class RegistroPonto {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne
    @JoinColumn(name = "funcionario_id")
    private Funcionario funcionario;

    private LocalDate data; // ex: 2025-01-15
    private LocalTime entrada; // ex: 08:00
    private LocalTime saida; // ex: 17:30
}
```

```
}
```

8. Criando os Repositories

Os **Repositories** são interfaces que o Spring implementa automaticamente. Você só precisa declarar a interface e o Spring cria todos os métodos de busca, salvar e deletar.

CargoRepository.java

```
package com.empresa.controlehoras.repository;

import com.empresa.controlehoras.model.Cargo;
import org.springframework.data.jpa.repository.JpaRepository;

// JpaRepository:
// Cargo = o tipo de entidade
// Long = tipo do ID (chave primária)
public interface CargoRepository
    extends JpaRepository<Cargo, Long> {

    // O Spring entende o nome do método e cria a query:
    // SELECT * FROM cargo WHERE nome = ?
    Cargo findByNome(String nome);
}
```

FuncionarioRepository.java

```
package com.empresa.controlehoras.repository;

import com.empresa.controlehoras.model.Funcionario;
import org.springframework.data.jpa.repository.JpaRepository;

public interface FuncionarioRepository
    extends JpaRepository<Funcionario, Long> {

    Funcionario findByMatricula(String matricula);
}
```

RegistroPontoRepository.java

```
package com.empresa.controlehoras.repository;

import com.empresa.controlehoras.model.Funcionario;
import com.empresa.controlehoras.model.RegistroPonto;
import org.springframework.data.jpa.repository.JpaRepository;
import java.time.LocalDate;
import java.util.List;
```

```
public interface RegistroPontoRepository
    extends JpaRepository<RegistroPonto, Long> {

    // Busca todos os registros de um funcionário em um período
    List<RegistroPonto> findByFuncionarioAndDataBetween(
        Funcionario funcionario,
        LocalDate inicio,
        LocalDate fim
    );
}
```

9. Criando os Services (Regras de Negócio)

Os **Services** contêm a lógica do negócio: calcular horas, processar o CSV, etc.

9.1 — HorasService.java — Cálculo das Horas

Este é o coração do sistema. Crie em `service/HorasService.java`:

```
package com.empresa.controlehoras.service;

import com.empresa.controlehoras.model.*;
import com.empresa.controlehoras.repository.*;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Service;
import java.time.*;
import java.util.*;

// @Service = indica que esta classe é um serviço Spring
// @RequiredArgsConstructor = Lombok injeta dependências pelo construtor
@Service
@RequiredArgsConstructor
public class HorasService {

    private final FuncionarioRepository funcionarioRepo;
    private final RegistroPontoRepository pontoRepo;

    /**
     * Calcula o relatório de horas de um funcionário
     * no período informado.
     */
    public Map<String, Object> calcularRelatorio(
        Long funcionarioId,
        LocalDate inicio,
        LocalDate fim) {

        // Busca o funcionário pelo ID
        Funcionario func = funcionarioRepo.findById(funcionarioId)
            .orElseThrow(() -> new RuntimeException(
                "Funcionário não encontrado: " + funcionarioId));
    }
}
```



```
// Busca os registros de ponto do período
List<RegistroPonto> registros =
    pontoRepo.findByFuncionarioAndDataBetween(
        func, inicio, fim);

double totalTrabalhado = 0;
double totalEsperado = 0;

for (RegistroPonto r : registros) {
    // Calcula horas trabalhadas no dia
    double horas = Duration.between(
        r.getEntrada(), r.getSaida()).toMinutes() / 60.0;
    totalTrabalhado += horas;

    // Horas esperadas vem do cargo do funcionário
    totalEsperado +=
        func.getCargo().getHorasDiarias();
}

double extras = Math.max(0,
    totalTrabalhado - totalEsperado);
double faltando = Math.max(0,
    totalEsperado - totalTrabalhado);

// Monta o resultado
Map<String, Object> resultado = new LinkedHashMap<>();
resultado.put("funcionario", func.getNome());
resultado.put("cargo", func.getCargo().getNome());
resultado.put("periodo",
    inicio + " a " + fim);
resultado.put("horasTrabalhadas",
    String.format("%.2f h", totalTrabalhado));
resultado.put("horasEsperadas",
    String.format("%.2f h", totalEsperado));
resultado.put("horasExtras",
    String.format("%.2f h", extras));
resultado.put("horasFaltando",
    String.format("%.2f h", faltando));
return resultado;
}
}
```

9.2 — CsvService.java — Importação de CSV

Responsável por ler o arquivo CSV e salvar os registros. Crie em **service/CsvService.java**:

```
package com.empresa.controlehoras.service;

import com.empresa.controlehoras.model.*;
import com.empresa.controlehoras.repository.*;
```

```
import com.opencsv.CSVReader;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Service;
import org.springframework.web.multipart.MultipartFile;
import java.io.InputStreamReader;
import java.time.LocalDate;
import java.time.LocalTime;
import java.util.ArrayList;
import java.util.List;

@Service
@RequiredArgsConstructor
public class CsvService {

    private final FuncionarioRepository funcRepo;
    private final RegistroPontoRepository pontoRepo;

    /**
     * Formato esperado do CSV:
     * matricula,data,entrada,saida
     * EMP001,2025-01-15,08:00,17:30
     */
    public String importarCsv(MultipartFile arquivo)
        throws Exception {

        int importados = 0;
        int erros      = 0;
        List<String> msgs = new ArrayList<>();

        try (CSVReader reader = new CSVReader(
            new InputStreamReader(
                arquivo.getInputStream()))) {

            String[] linha;
            boolean primeiraLinha = true;

            while ((linha = reader.readNext()) != null) {
                // Pula o cabeçalho
                if (primeiraLinha) {
                    primeiraLinha = false;
                    continue;
                }

                try {
                    String matricula = linha[0].trim();
                    LocalDate data =
                        LocalDate.parse(linha[1].trim());
                    LocalTime entrada =
                        LocalTime.parse(linha[2].trim());
                    LocalTime saida =
                        LocalTime.parse(linha[3].trim());
```

```
        Funcionario func =
            funcRepo.findByMatricula(matricula);
        if (func == null) {
            erros++;
            msgs.add("Matrícula não encontrada: "
                + matricula);
            continue;
        }

        RegistroPonto reg = new RegistroPonto();
        reg.setFuncionario(func);
        reg.setData(data);
        reg.setEntrada(entrada);
        reg.setSaida(saida);
        pontoRepo.save(reg);
        importados++;

    } catch (Exception e) {
        erros++;
        msgs.add("Erro na linha: " + e.getMessage());
    }
}

return importados + " registros importados, "
    + erros + " erros. " + msgs;
}
}
```

10. Criando os Controllers (API REST)

Os **Controllers** definem as URLs que o sistema responde. Cada método vira um endpoint da API.

10.1 — FuncionarioController.java

```
package com.empresa.controlehoras.controller;

import com.empresa.controlehoras.model.*;
import com.empresa.controlehoras.repository.*;
import lombok.RequiredArgsConstructor;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import java.util.List;

// @RestController = controller que retorna JSON
// @RequestMapping = prefixo de URL para todos os métodos
@RestController
@RequestMapping("/api/funcionarios")
@RequiredArgsConstructor
```

```
public class FuncionarioController {

    private final FuncionarioRepository funcRepo;
    private final CargoRepository cargoRepo;

    // GET /api/funcionarios – lista todos
    @GetMapping
    public List<Funcionario> listar() {
        return funcRepo.findAll();
    }

    // POST /api/funcionarios – cadastra novo
    @PostMapping
    public ResponseEntity<Funcionario> cadastrar(
        @RequestBody Funcionario func) {
        return ResponseEntity.ok(
            funcRepo.save(func));
    }

    // DELETE /api/funcionarios/{id} – remove
    @DeleteMapping("/{id}")
    public ResponseEntity<Void> deletar(
        @PathVariable Long id) {
        funcRepo.deleteById(id);
        return ResponseEntity.noContent().build();
    }
}
```

10.2 — CargoController.java

```
package com.empresa.controlehoras.controller;

import com.empresa.controlehoras.model.Cargo;
import com.empresa.controlehoras.repository.CargoRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequestMapping("/api/cargos")
@RequiredArgsConstructor
public class CargoController {

    private final CargoRepository cargoRepo;

    @GetMapping
    public List<Cargo> listar() {
        return cargoRepo.findAll();
    }
}
```

```
@PostMapping
public ResponseEntity<Cargo> criar(
    @RequestBody Cargo cargo) {
    return ResponseEntity.ok(
        cargoRepo.save(cargo));
}

@PutMapping("/{id}")
public ResponseEntity<Cargo> atualizar(
    @PathVariable Long id,
    @RequestBody Cargo cargo) {
    cargo.setId(id);
    return ResponseEntity.ok(
        cargoRepo.save(cargo));
}
}
```

10.3 — PontoController.java

```
package com.empresa.controlehoras.controller;

import com.empresa.controlehoras.service.*;
import lombok.RequiredArgsConstructor;
import org.springframework.format.annotation.DateTimeFormat;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.multipart.MultipartFile;
import java.time.LocalDate;
import java.util.Map;

@RestController
@RequestMapping("/api/ponto")
@RequiredArgsConstructor
public class PontoController {

    private final CsvService csvService;
    private final HorasService horasService;

    // POST /api/ponto/importar – envia o arquivo CSV
    @PostMapping("/importar")
    public ResponseEntity<String> importar(
        @RequestParam("arquivo") MultipartFile file)
        throws Exception {
        return ResponseEntity.ok(
            csvService.importarCsv(file));
    }

    // GET /api/ponto/relatorio?funcionarioId=1
    //      &inicio;=2025-01-01&fim;=2025-01-31
    @GetMapping("/relatorio")
    public ResponseEntity<Map<String, Object>> relatorio(
```

```
        @RequestParam Long funcionarioId,
        @RequestParam
        @DateTimeFormat(iso = DateTimeFormat.ISO.DATE)
        LocalDate inicio,
        @RequestParam
        @DateTimeFormat(iso = DateTimeFormat.ISO.DATE)
        LocalDate fim) {
    return ResponseEntity.ok(
        horasService.calcularRelatorio(
            funcionarioId, inicio, fim));
}
```

11. Formato do Arquivo CSV

O arquivo CSV deve seguir exatamente este formato. Crie um arquivo chamado **ponto.csv** para testar:

```
matricula,data,entrada,saida
EMP001,2025-01-13,08:00,17:30
EMP001,2025-01-14,08:15,18:00
EMP001,2025-01-15,07:50,17:00
EMP002,2025-01-13,09:00,18:30
EMP002,2025-01-14,08:00,17:00
EMP003,2025-01-13,08:00,20:00
```

matricula	Matrícula do funcionário (deve existir no sistema)
data	Data no formato AAAA-MM-DD (ex: 2025-01-13)
entrada	Horário de entrada no formato HH:MM (ex: 08:00)
saida	Horário de saída no formato HH:MM (ex: 17:30)

■ O cabeçalho (primeira linha) é obrigatório e deve ser exatamente: `matricula,data,entrada,saida`

12. Rodando e Testando o Sistema

12.1 — Iniciando o servidor

- No IntelliJ, abra **ControleHorasApplication.java**
- Clique no triângulo verde ■ ao lado do método **main**
- Aguarde aparecer no console: **Started ControleHorasApplication in X.XXX seconds**

12.2 — Testando pelo navegador (GET)

Abra o navegador e acesse:

```
# Listar todos os cargos
http://localhost:8080/api/cargos

# Listar todos os funcionários
http://localhost:8080/api/funcionarios

# Relatório de horas (substitua os valores)
http://localhost:8080/api/ponto/relatorio
?funcionarioId=1&inicio;=2025-01-01&fim;=2025-01-31
```

12.3 — Cadastrando dados com Postman ou curl

Instale o **Postman** (<https://www.postman.com>) para fazer requisições POST. Você também pode usar o **curl** no terminal:

```
# 1. Criar um cargo
curl -X POST http://localhost:8080/api/cargos \
  -H "Content-Type: application/json" \
  -d '{"nome":"Desenvolvedor","horasDiarias":8.0}'

# 2. Criar um funcionário (use o id do cargo criado acima)
curl -X POST http://localhost:8080/api/funcionarios \
  -H "Content-Type: application/json" \
  -d '{"nome":"João Silva","matricula":"EMP001",
    "cargo":{"id":1}}'

# 3. Importar CSV
curl -X POST http://localhost:8080/api/ponto/importar \
  -F "arquivo=@ponto.csv"

# 4. Ver relatório
curl "http://localhost:8080/api/ponto/relatorio\
?funcionarioId=1&inicio;=2025-01-01&fim;=2025-01-31"
```

Resposta esperada do relatório:

```
{
  "funcionario": "João Silva",
  "cargo": "Desenvolvedor",
  "periodo": "2025-01-01 a 2025-01-31",
  "horasTrabalhadas": "25.50 h",
  "horasEsperadas": "24.00 h",
  "horasExtras": "1.50 h",
  "horasFaltando": "0.00 h"
}
```

13. Visualizando o Banco de Dados (Console H2)

O H2 oferece um console visual para ver os dados no banco enquanto o sistema roda:

- Acesse: <http://localhost:8080/h2-console>

- JDBC URL: **jdbc:h2:mem:controlehoras**
- User Name: **sa** | Password: (deixe em branco)
- Clique em **Connect**

Você verá as tabelas **CARGO**, **FUNCIONARIO** e **REGISTRO_PONTO**. Clique em qualquer tabela e depois em **Run** para ver os dados.

■ Este console é perfeito para depurar e verificar se os dados foram salvos corretamente após importar o CSV.

14. Próximos Passos e Melhorias

Parabéns! Você tem um sistema funcional. Agora pode evoluir com:

Banco de dados persistente

Troque o H2 pelo **PostgreSQL** ou **MySQL** para não perder os dados ao reiniciar.

```
# No pom.xml, adicione:
<dependency>
  <groupId>org.postgresql</groupId>
  <artifactId>postgresql</artifactId>
</dependency>

# No application.properties, troque:
spring.datasource.url=jdbc:postgresql://localhost:5432/controlehoras
spring.datasource.username=seu_usuario
spring.datasource.password=sua_senha
```

Interface Web

- Adicione o Thymeleaf para criar páginas HTML com formulários
- Ou crie um frontend em React/Vue que consuma sua API

Segurança

- Adicione Spring Security para proteger os endpoints com login e senha
- Implemente autenticação JWT para acesso via token

Relatórios em PDF

- Use a biblioteca iText ou JasperReports para gerar relatórios em PDF

■ Você já tem uma base sólida em Java e Spring Boot. Continue praticando!

15. Glossário Rápido

Spring Boot	Framework Java que facilita criar aplicações web sem configuração complexa
API REST	Interface que permite que sistemas se comuniquem via HTTP (GET, POST, PUT, DELETE)
JPA / Hibernate	Biblioteca que mapeia classes Java para tabelas do banco de dados
H2	Banco de dados leve que roda na memória RAM (ótimo para testes)
Lombok	Biblioteca que gera automaticamente getters, setters e construtores
Maven	Gerenciador de dependências: baixa e organiza as bibliotecas do projeto
Entity	Classe Java que representa uma tabela no banco de dados
Repository	Interface que abstrai o acesso ao banco de dados
Service	Classe com a lógica de negócio da aplicação
Controller	Classe que recebe requisições HTTP e retorna respostas JSON
CSV	Arquivo de texto onde os valores são separados por vírgula
JSON	Formato de dados usado para troca de informações entre sistemas
Endpoint	URL específica da API que realiza uma ação